

01_database-py

#PostgreSQL #python

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
├── backend/  
│   ├── app/  
│   │   ├── db/  
│   │   │   └── database.py
```

El archivo `database.py` se encuentra dentro del módulo de base de datos del backend:

```
backend/app/db/
```

Su función principal es configurar la conexión entre el backend FastAPI y la base de datos PostgreSQL mediante SQLAlchemy.

Este archivo define tres elementos importantes:

```
DATABASE_URL → cadena de conexión con PostgreSQL  
engine        → motor de conexión de SQLAlchemy  
SessionLocal  → fábrica de sesiones de base de datos
```

También incluye una función auxiliar para comprobar la conexión:

```
test_db_connection()
```

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,220p' backend/app/db/database.py
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`, utilizado para leer o transformar texto.

```
-n
```

Evita que `sed` imprima todo el archivo automáticamente.

```
'1,220p'
```

Indica que se impriman las líneas de la 1 a la 220.

```
backend/app/db/database.py
```

Es la ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```
import os
from sqlalchemy import create_engine, text
from sqlalchemy.orm import sessionmaker

DATABASE_URL = os.getenv("DATABASE_URL")
if not DATABASE_URL:
    raise RuntimeError("DATABASE_URL no está definido")

engine = create_engine(DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)

def test_db_connection() -> None:
    with engine.connect() as conn:
        conn.execute(text("SELECT 1"))
```

4 Función general del archivo

El archivo `database.py` configura la conexión principal del backend con PostgreSQL.

Su responsabilidad no es crear endpoints ni definir tablas, sino preparar la infraestructura necesaria para que otros archivos puedan trabajar con la base de datos.

A nivel técnico, este archivo hace lo siguiente:

1. Importa herramientas necesarias.
2. Lee la variable de entorno `DATABASE_URL`.
3. Comprueba que `DATABASE_URL` existe.
4. Crea el motor de conexión SQLAlchemy.
5. Crea una fábrica de sesiones.
6. Define una función para probar la conexión.

La relación general es:

```
docker/.env
  ↓
DATABASE_URL
  ↓
database.py
  ↓
create_engine()
  ↓
SessionLocal
  ↓
get_db()
  ↓
endpoints FastAPI
  ↓
PostgreSQL
```

Este archivo es fundamental porque sin `engine` ni `SessionLocal`, los endpoints no podrían consultar ni modificar la base de datos.

5 Estructura general del archivo

El archivo puede dividirse en cinco bloques:

```
import os
from sqlalchemy import create_engine, text
from sqlalchemy.orm import sessionmaker
```

Importaciones necesarias.

```
DATABASE_URL = os.getenv("DATABASE_URL")
```

Lectura de la variable de entorno con la cadena de conexión.

```
if not DATABASE_URL:
    raise RuntimeError("DATABASE_URL no está definido")
```

Validación de que la variable existe.

```
engine = create_engine(DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)
```

Creación del motor de conexión y de la fábrica de sesiones.

```
def test_db_connection() -> None:
    with engine.connect() as conn:
        conn.execute(text("SELECT 1"))
```

Función auxiliar para comprobar la conexión.

Visualmente:

```
database.py
├─ Importaciones
├─ Lectura de DATABASE_URL
├─ Validación de DATABASE_URL
├─ Creación de engine
├─ Creación de SessionLocal
└─ Función test_db_connection()
```

6 Análisis línea por línea

Importación del módulo `os`

```
import os
```

Esta línea importa el módulo estándar `os` de Python.

El módulo `os` permite interactuar con el sistema operativo y con el entorno de ejecución.

En este archivo se utiliza para leer variables de entorno:

```
os.getenv("DATABASE_URL")
```

Una variable de entorno es un valor configurado fuera del código. En este proyecto, `DATABASE_URL` se define desde los archivos `.env` y se pasa al contenedor mediante Docker Compose.

Esto permite que la cadena de conexión no esté escrita directamente en el código.

Importación de `create_engine` y `text`

```
from sqlalchemy import create_engine, text
```

Esta línea importa dos elementos desde SQLAlchemy:

```
create_engine
text
```

create_engine

`create_engine` se utiliza para crear el motor de conexión con la base de datos.

El motor de SQLAlchemy es el objeto que sabe cómo conectarse a PostgreSQL usando una cadena de conexión.

En este archivo se usa aquí:

```
engine = create_engine(DATABASE_URL, pool_pre_ping=True)
```

text

`text` permite escribir una consulta SQL manual como texto y convertirla en una expresión ejecutable por SQLAlchemy.

En este archivo se usa aquí:

```
conn.execute(text("SELECT 1"))
```

Esto permite ejecutar una consulta simple para comprobar que la base de datos responde.

Importación de sessionmaker

```
from sqlalchemy.orm import sessionmaker
```

Esta línea importa `sessionmaker` desde el módulo ORM de SQLAlchemy.

`sessionmaker` permite crear una fábrica de sesiones.

Una sesión de SQLAlchemy es el objeto que se utiliza para trabajar con la base de datos desde Python.

A través de una sesión se pueden realizar operaciones como:

- Consultar registros.
- Insertar datos.
- Actualizar datos.
- Eliminar datos.
- Ejecutar consultas SQL.

En este archivo se usa aquí:

```
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)
```

Lectura de la variable DATABASE_URL

```
DATABASE_URL = os.getenv("DATABASE_URL")
```

Esta línea lee la variable de entorno `DATABASE_URL`.

Desglose:

```
DATABASE_URL
```

Nombre de la variable Python donde se guardará el valor.

```
=
```

Operador de asignación.

```
os.getenv("DATABASE_URL")
```

Llama a la función `getenv` del módulo `os`.

`os.getenv` busca una variable de entorno con el nombre indicado.

En este caso busca:

```
DATABASE_URL
```

Si la variable existe, devuelve su valor.

Si no existe, devuelve `None`.

Una cadena típica de conexión en este proyecto tiene esta forma:

```
postgresql+psycopg://siem:change_me@db:5432/siem
```

Desglose conceptual:

```
postgresql+psycopg → dialecto y driver
siem                → usuario
change_me           → contraseña
db                  → host del servicio PostgreSQL en Docker
5432                → puerto interno de PostgreSQL
siem                → nombre de la base de datos
```

Validación de `DATABASE_URL`

```
if not DATABASE_URL:
```

Esta línea comprueba si `DATABASE_URL` está vacía o no existe.

Desglose:

```
if
```

Palabra clave de Python para crear una condición.

```
not DATABASE_URL
```

Significa “si `DATABASE_URL` no tiene valor”.

Esto será verdadero si `DATABASE_URL` es:

```
None
cadena vacía
valor falso
```

Esta comprobación es importante porque el backend no puede funcionar sin la cadena de conexión a PostgreSQL.

Excepción si falta `DATABASE_URL`

```
raise RuntimeError("DATABASE_URL no está definido")
```

Esta línea lanza un error si la variable `DATABASE_URL` no está definida.

Desglose:

```
raise
```

Palabra clave de Python para lanzar una excepción.

```
RuntimeError(...)
```

Tipo de excepción. Se utiliza para indicar un error en tiempo de ejecución.

```
"DATABASE_URL no está definido"
```

Mensaje del error.

Esto significa que si el backend arranca sin `DATABASE_URL`, la aplicación fallará de forma explícita.

Es mejor fallar con un mensaje claro que intentar arrancar y provocar errores más confusos después.

Ejemplo de problema que evita:

```
Backend arrancando sin saber dónde está PostgreSQL.  
Endpoints fallando después con errores de conexión poco claros.
```

Con esta validación, el error aparece al inicio y es más fácil de diagnosticar.

Creación del `engine`

```
engine = create_engine(DATABASE_URL, pool_pre_ping=True)
```

Esta línea crea el motor de conexión de SQLAlchemy.

Desglose:

```
engine
```

Variable donde se guarda el motor.

```
=
```

Operador de asignación.

```
create_engine(...)
```

Función de SQLAlchemy que crea un motor de conexión.

```
DATABASE_URL
```

Cadena de conexión con PostgreSQL.

```
pool_pre_ping=True
```

Opción de SQLAlchemy para comprobar que una conexión está viva antes de usarla.

Qué es el `engine`

El `engine` es uno de los objetos centrales de SQLAlchemy.

No representa una tabla ni una consulta concreta. Representa la configuración de conexión con la base de datos.

Se puede entender como:

```
engine → punto de acceso principal a PostgreSQL
```

A partir del `engine`, SQLAlchemy puede abrir conexiones y crear sesiones.

Qué hace `pool_pre_ping=True`

SQLAlchemy utiliza un pool de conexiones.

Un pool de conexiones mantiene conexiones reutilizables con la base de datos para no tener que abrir una conexión nueva en cada operación.

El problema es que, a veces, una conexión del pool puede quedar cerrada o inválida.

La opción:

```
pool_pre_ping=True
```

hace que SQLAlchemy compruebe si la conexión está viva antes de usarla.

Si detecta que no lo está, intenta renovarla.

Esto ayuda a evitar errores por conexiones antiguas o caducadas.

En un laboratorio con Docker, esto puede ser útil porque los contenedores pueden reiniciarse o la base de datos puede tardar en estar disponible.

Creación de SessionLocal

```
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)
```

Esta línea crea una fábrica de sesiones de base de datos.

Desglose:

```
SessionLocal
```

Nombre de la variable.

Se usa una convención habitual en proyectos FastAPI + SQLAlchemy: llamar `SessionLocal` a la fábrica de sesiones.

```
=
```

Operador de asignación.

```
sessionmaker(...)
```

Función de SQLAlchemy que crea una fábrica de sesiones.

Una fábrica de sesiones no es una sesión concreta. Es un objeto que permite crear sesiones cuando se necesitan.

Parámetro bind=engine

```
bind=engine
```

Indica que las sesiones creadas por `SessionLocal` estarán vinculadas al `engine`.

Es decir:

```
SessionLocal usa el engine para conectarse a PostgreSQL.
```

La relación sería:

```
DATABASE_URL
  ↓
engine
  ↓
SessionLocal
  ↓
db session
```

Parámetro `autoflush=False`

```
autoflush=False
```

Controla si SQLAlchemy debe enviar automáticamente cambios pendientes a la base de datos antes de ciertas consultas.

Con `autoflush=False`, el desarrollador tiene mayor control sobre cuándo se sincronizan los cambios.

Esto evita que SQLAlchemy haga operaciones implícitas en momentos no esperados.

En un proyecto pequeño o MVP, esta configuración ayuda a mantener el comportamiento más explícito.

Parámetro `autocommit=False`

```
autocommit=False
```

Indica que las transacciones no se confirman automáticamente.

Esto significa que, cuando se hagan cambios en la base de datos, normalmente será necesario llamar explícitamente a:

```
db.commit()
```

Esto es una buena práctica porque permite controlar cuándo se guardan definitivamente los cambios.

Si algo falla antes del `commit`, se puede hacer rollback o evitar que los datos queden parcialmente guardados.

Separación antes de la función

La línea en blanco separa la configuración inicial de la definición de funciones.

No afecta al funcionamiento del programa.

Sirve para mejorar la legibilidad del archivo.

Definición de `test_db_connection`

```
def test_db_connection() -> None:
```

Esta línea define una función llamada `test_db_connection`.

Desglose:

```
def
```

Palabra clave de Python para definir una función.

```
test_db_connection
```

Nombre de la función.

```
()
```

La función no recibe parámetros.

```
-> None
```

Anotación de tipo que indica que la función no devuelve ningún valor.

```
:
```

Indica el inicio del bloque de código de la función.

La función sirve para comprobar que la conexión con la base de datos funciona.

Apertura de conexión con `engine.connect`

```
with engine.connect() as conn:
```

Esta línea abre una conexión directa usando el `engine`.

Desglose:

```
with
```

Palabra clave de Python para usar un gestor de contexto.

Un gestor de contexto se encarga de abrir y cerrar recursos correctamente.

```
engine.connect()
```

Abre una conexión con la base de datos usando el motor SQLAlchemy.

```
as conn
```

Guarda la conexión abierta en la variable `conn`.

El uso de `with` garantiza que la conexión se cierre correctamente al salir del bloque, incluso si ocurre un error.

Ejecución de consulta de prueba

```
conn.execute(text("SELECT 1"))
```

Esta línea ejecuta una consulta SQL sencilla usando la conexión abierta.

Desglose:

```
conn
```

Variable que representa la conexión activa.

```
.execute(...)
```

Método para ejecutar una instrucción SQL.

```
text("SELECT 1")
```

Convierte la cadena `"SELECT 1"` en una expresión SQL ejecutable por SQLAlchemy.

Consulta `SELECT 1`

```
SELECT 1
```

Es una consulta mínima que no depende de ninguna tabla.

Su objetivo es comprobar si la base de datos responde.

Si PostgreSQL está disponible y la conexión es correcta, la consulta se ejecuta sin errores.

Si hay un problema de conexión, credenciales, red o configuración, esta línea lanzará una excepción.

Resultado final del archivo

Después de cargar este archivo, quedan definidos:

```
DATABASE_URL
engine
SessionLocal
test_db_connection()
```

Estos elementos son reutilizados por otros módulos del backend.

La relación principal es:

```
database.py
↓
SessionLocal
↓
session.py
↓
get_db()
↓
endpoints FastAPI
```

7 Relación con el flujo técnico del laboratorio

Este archivo conecta el backend con PostgreSQL.

No define endpoints ni modelos, pero proporciona la infraestructura necesaria para que otros módulos puedan operar con la base de datos.

La relación técnica sería:

```
Docker Compose
↓
DATABASE_URL
↓
database.py
↓
engine
↓
SessionLocal
↓
get_db()
↓
endpoints
↓
PostgreSQL
```

Dentro del flujo general del SIEM:

```
Evento recibido por la API
↓
Endpoint usa get_db()
↓
get_db() crea sesión desde SessionLocal
↓
SessionLocal usa engine
↓
engine conecta con PostgreSQL
↓
evento se almacena en la base de datos
```

Sin `database.py`, el backend no tendría una forma centralizada de conectarse a PostgreSQL.

8 Errores típicos o puntos importantes

Falta DATABASE_URL

Si la variable de entorno no está definida, se ejecuta:

```
raise RuntimeError("DATABASE_URL no está definido")
```

Esto detiene el arranque de la aplicación.

Solución:

- Revisar .env.
- Revisar docker/.env.
- Revisar docker/compose.yml.
- Comprobar que DATABASE_URL se pasa al servicio api.

Uso incorrecto de localhost dentro de Docker

Dentro del contenedor siem-api, la base de datos no debe referenciarse normalmente como:

```
localhost
```

porque localhost dentro del contenedor apunta al propio contenedor de la API.

En Docker Compose, lo correcto es usar el nombre del servicio:

```
db
```

Por eso una cadena válida suele ser:

```
postgresql+psycpg://siem:change_me@db:5432/siem
```

PostgreSQL no está disponible

Aunque DATABASE_URL exista, la conexión puede fallar si:

- El contenedor siem-db está apagado.
- PostgreSQL todavía no está listo.
- Las credenciales son incorrectas.
- La red Docker no está bien configurada.
- El puerto o nombre del host no son correctos.

Diferencia entre engine y SessionLocal

engine no es una sesión.

```
engine → configura la conexión con la base de datos
SessionLocal → crea sesiones de trabajo
session/db → se usa dentro de endpoints para consultar o modificar datos
```

Confundir estos conceptos puede llevar a errores de diseño.

test_db_connection no se ejecuta automáticamente

La función:

```
test_db_connection()
```

queda definida, pero no se ejecuta por sí sola.

Solo se ejecutará si algún otro archivo la llama explícitamente.

9 Comandos útiles relacionados

Comprobar si `DATABASE_URL` está definida dentro del contenedor:

```
docker exec -it siem-api env | grep DATABASE_URL
```

Probar importación de `database.py` :

```
docker exec -it siem-api python -c "from app.db.database import engine, SessionLocal; print(engine)"
```

Ejecutar la función de prueba:

```
docker exec -it siem-api python -c "from app.db.database import test_db_connection; test_db_connection(); print('db ok')"
```

Comprobar conexión directa desde PostgreSQL:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT 1;"
```

Ver contenedores activos:

```
docker ps
```

Ver logs de la API:

```
docker logs siem-api
```

Ver logs de PostgreSQL:

```
docker logs siem-db
```

Levantar el entorno:

```
docker compose --env-file docker/.env -f docker/compose.yml up -d
```

Reiniciar solo la API:

```
docker compose --env-file docker/.env -f docker/compose.yml restart api
```